

PHP コーディング規約

Ver. 1.0.1 [2013/02/08]

改定履歴

Ver. 1.0.0 2012/09/15

初版

Ver. 1.0.1 2013/02/08

インデント」「制御構造」を追加

目次

1	命名規則	4
1.1	関数	4
1.2	グローバル/ローカル変数	4
1.3	配列	5
1.4	オブジェクト変数	6
1.5	定数	6
1.6	クラス名	6
2	PHP コードタグ	7
3	関数のコール	7
4	コメント表記	7
4.1	ファイルのヘッダーのコメントブロック	7
4.2	関数、クラスメソッドのコメントブロック	8
5	PHP ファイルの読み込み（インクルード）	8
6	インデント	8
7	制御構造	9
7.1	If/Else/Elseif	9
7.2	Switch	10
8	参考	11

1 命名規則

1.1 関数

『動詞』または『動詞＋名詞』となるように命名し、先頭文字（動詞）は、英小文字からはじめる。「動詞」の部分は必ず英単語とする。「名詞」の部分に関しては、英単語、あるいは日本語ローマ字とする。

また、『動詞』の部分が長い英単語の時は、初めの数文字で短縮することも可能とする。しかし、名詞の部分等は、省略しすぎないように注意。関数名を短くする事より、短く略したが為に、他人が見て何をする関数か判断できなくなる危険性に注意する。

※関数名にアンダーバー（`_`）は使用しない。

例 1）計算結果を取得する関数の場合：

`getResult()` または、`getKekka()` 等・・・

例 2）値段を設定する関数の場合：

`setPrice()` または、`setNedan()` 等・・・

例 3）ポイントを計算する関数の場合：

`calcPoints()` 等・・・

`*calc = calculation[計算]`

例 4）日付のフォーマットを変換する場合：

`convDateFormat()`

`*conv = convert[変換]`

例 5）名前を出力・表示(echo)する関数の場合：

`showName()` または、`echoName()` 等・・・

例 6）管理者 (Administrator) だと TRUE、違う場合は FALSE を返す関数

`isAdmin()`

*『`〇〇〇 is Administrator.`』を略して `is Admin` → `isAdmin()`

1.2 グローバル/ローカル変数

関数名と同じく、省略して短くする事より、省略した事で他人が見て何の変数か判断できなくなる危険性を考慮する。その為に変数名が長くなる事は問題としない。

A) 変数命名の基本

変数名は、1 語以上の単語で構成し、必ず名詞（又は形容詞＋名詞）とする。先頭文字は小文字にし、2 つ以上の単語で構成される場合は、2 つ目からの単語の頭文字は大文字にする。またアンダーバーは、使用しない。

例) \$name、\$customerName、\$userId、\$soldProductId、\$productNumber

B) ループ内カウントアップ/ダウン用変数

for 文等のループで使用するカウントアップ/ダウン用の変数等は、i や n、cnt (count の略) 等の小文字アルファベット 1 ～ 3 つで表すことが可能。

ただし、ループ内の処理が何十行にも及ぶ長いプログラムの場合や、多重ループの場合は、「i」「cnt」の後に名詞（英単語名詞、あるいは日本語のローマ字）をつける。

※多重ループの場合、処理が短いループ以外は、出来る限り \$i2、\$i3 等のアルファベット 1 文字に数値をつける命名は避ける。

例)

- ・ ループ時のカウント用変数の基本例

\$i、\$n、\$cnt

- ・ ループ内の処理が長い場合の例

\$iUsers、\$iRows、\$cntProducts

1.3 配列

- ・ 先頭に『arr』をつける時

配列の場合は、配列名の先頭に『arr』をつける。

例)

\$arrUserIDs、\$arrProductNames

- ・ 先頭に『arr』をつけない時

DB からのデータ抽出結果等、先頭に「arr」を付けずに命名することも可能とする。

但し、その場合は、必ず名詞の複数形（語尾に「s」をつける）にする

例)

\$rows、\$lines、\$results

【参考】

○複数形の基本形

例 : Friend→Friends / Car→Cars

○単語が s, sh, ch, x, o で終わっている場合は es を付ける

例 : Fish→Fishes / Watch→Watches

○子音+y で終わっている単語は、y を i に変えて es を付けます。

例 : Baby→Babys ※母音+y の場合は s を付けるだけ。(joy→joys)

1.4 オブジェクト変数

ユーザ定義のクラスをインスタンス化したオブジェクトを格納する変数は、先頭に『obj』をつける。

例) \$objCsvReader、\$objFileWriter

ただし、データベースアクセスインターフェース等のユーザ定義ではないオブジェクト変数は、この限りではない。

例) \$dbi *DB アクセスインターフェースの例

1.5 定数

define() やクラス内で const で定義する定数は、全て大文字で、単語と単語の間にはアンダーバー (_) を使用する。単語は英単語、あるいは日本語ローマ字で表し、あまり省略し過ぎないように注意する。

例) DB_LOGIN_ID、PRINTOUT_LEFT_MARGIN、ADMIN_USER_NAME

1.6 クラス名

クラス名は、先頭文字を含めて、個々の単語の頭文字は全て大文字にする。また内容が理解できるように、略語の使用は可能な限り避ける。

また、クラス名は基本的に名詞とする。

例) LogReader、FileAccesssser、CsvFileWriter

クラスメソッド（クラス関数）

「[関数](#)」の命名規則と同じ。

クラス変数（メンバー変数）

「[グローバル/ローカル変数](#)」の命名規則と同じ

2 PHP コードタグ

PHP ファイル内の PHP コード領域は、必ず `<?php ?>` で囲む。

※「php」の部分を略した `<? ?>` で囲まない。これはルールとしてだけでなく、異なるサーバー環境であっても、PHP のスクリプトとして認識される最も互換性の高い方法であり意義がある。

【重要】

また、ファイルの最後が PHP コード領域の終端 `?>` で終わる場合は、`?>` の後にスペースや改行を入れてはいけない。

3 関数のコール

関数をコール（呼び出し）する時、パラメータのある場合は、開きカッコと最初のパラメータの間に空白を入れない。また最後のパラメータと閉じカッコの間も空白を入れない。

間違った例：

```
$result = fooFunc( $param ); // ← $param の両端に空白が入っているのは×
```

正しい例：

```
$result = fooFunc($param);
```

関数に渡されるパラメータが複数の場合は、カンマと次のパラメータの間に空白を入れる。

正しい例：

```
$result = fooFunc($param1, $param2, $param3);
```

4 コメント表記

4.1 ファイルのヘッダーのコメントブロック

PHP ファイルのヘッダー（最初の行から数行）には、コメントブロックを用いてファイルの説明を入れる。

また `@author`（作成者）や `@since`（更新日）を表記する。表記する内容は、各プロジェクトで決定し、統一する。

例：

```
/**
```

```
 * DB アクセス関数群
```

```
*
* @author 山田 太郎(yamada@fine-system.co.jp) ※作成/編集者
* @since 2012/09/01 ※更新日
*/
```

4.2 関数、クラスメソッドのコメントブロック

関数、クラスメソッドの上に、コメントブロックを使用して、関数の説明、パラメータの説明 (@param)、戻り値の説明 (@return) を記入する。

例：

```
/**
 * ファイル読み込み関数
 * @param string ファイルパス
 * @param string ファイル名
 * @return boolean TRUE:成功, FALSE:失敗
 */
Function readFile($path, $filename) {
    ***** 略 *****
    return $result;
}
```

5 PHP ファイルの読み込み(インクルード)

PHP ファイルから、他の PHP ファイルをインクルードする場合は、特別な理由がある時以外は、require()、又は require_once() を用いる。

但し、読み込むファイルが存在しない場合も考慮されたプログラムの場合に限っては、include()、又は include_once() を用いる事が可能。

6 インデント

プログラム内のインデントは、半角スペース4つつ分、或いはタブ1つ文とする。半角スペースかタブを使うかの判断は、プロジェクト単位で決めて統一する。タブの場合は、エディターの表示設定を半角4文字分を表示するように設定する。

7 制御構造

7.1 If/Else/Elseif

「if」と条件を指定するカッコ「(」の間にはスペースを入れず、条件指定のカッコを閉じた後も波カッコとの間に空白を入れない。また、なるべく if の前に条件を説明するコメントを入れる。

例 1)

```
// ユーザー入力の値が 100 の時
if($a == 100) {
    $b = '満点' ;
}
```

例 2)

```
// ユーザー入力の値が 100 の時
if($a == 100)
{
    $b = '満点' ;
}
```

条件分が複数あり、1 行の文字数が長くなる場合は、論理演算子の前で改行を入れ、条件句の最初の文字がそろうようにインデントする。

例)

```
// ユーザー入力値が必要条件を全て満たすとき
if(($a == 100)
    && ($b == 20000)
    && ($c == 50000)
    || ($d != 70000)
){
    $result = 10000;
}
```

else や elseif は、その前の閉じる波カッコ「)」で改行を入れてから記入する。また、「)」と else/elseif の間に条件を説明するコメントを入れる。

例)

```
// ユーザー入力の値が 100 の時
if($a == 100) {
    $result = true;
```

```

}
// ユーザー入力の値が 200 の時
elseif($b == 200) {
    $result = false;
}
// ユーザー入力の値が 100 でも 200 でもない時
else{
    changeValue($a);
    $result = false;
}

```

7.2 Switch

Switch 文では、各 case 文をインデントし、また case 文の中身をさらにインデントする。また、case 文の前に条件を説明するコメントを入れる。

例)

```

switch($numRate) {
    // 100 の時
    case 100:
        echo 'OK' ;
        break;
    // 1000 の時
    case 1000:
        echo 'enough' ;
        break;
    // 10000 の時
    case 10000:
        echo " too much;
        break;
    // 指定値以外るとき
    default:
        echo 'unknown' :
}

```

8 参考

この PHP のコーディング・命名規則は、Zend Framework、PEAR で定義されている命名規則を元に作られています。

- ・ PERE :: Manual :: 命名規約

<http://pear.php.net/manual/ja/standards.naming.php>

- ・ Zend Framework: Documentation

<http://framework.zend.com/manual/ja/coding-standard.naming-conventions.html>